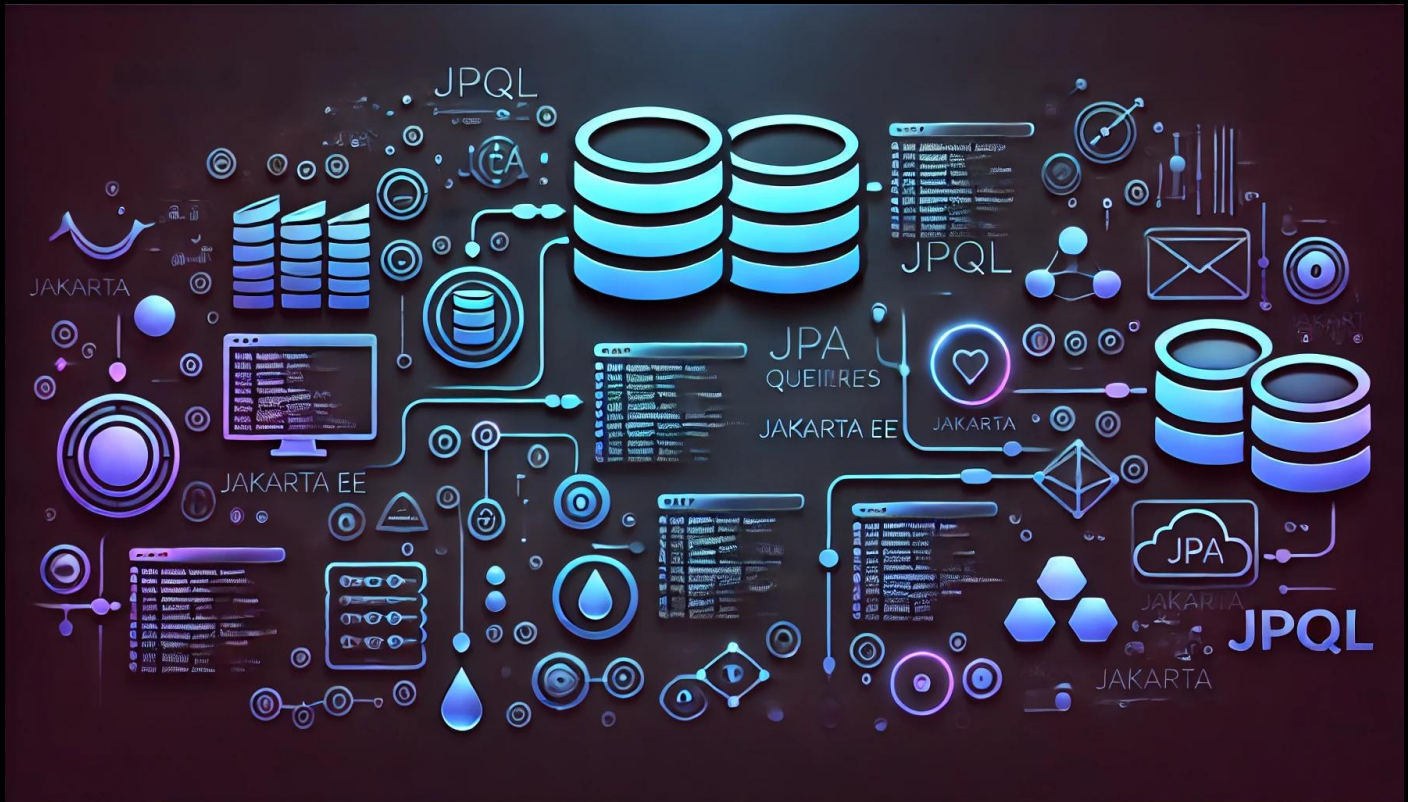


JPQL con JPA y Jakarta EE – parte 5



JPQL con JPA en Jakarta EE – parte 5

En esta guía aprenderemos a realizar **consultas avanzadas con JPQL en JPA**, enfocándonos en dos conceptos clave:

1. **Recuperar solo los IDs de todas las personas** en la base de datos.
2. **Utilizar funciones escalares** para obtener el **valor mínimo, máximo y el conteo de registros**.

Este tipo de consultas son útiles para optimizar el rendimiento al trabajar solo con los datos necesarios.

✦ Ejemplo práctico:

- Queremos obtener **solo los IDs** de las personas en la base de datos.
- Queremos calcular el **ID más pequeño, el más grande y el total de registros** en la tabla `Persona`.

1 Conceptos Básicos

✦ ¿Qué es JPQL y cómo funcionan las consultas escalares?

JPQL permite realizar consultas similares a SQL pero con **entidades en lugar de tablas**.

Ejemplo en **SQL**:

```
SELECT id_persona FROM persona;
SELECT MIN(id_persona), MAX(id_persona), COUNT(id_persona) FROM persona;
```

Equivalente en **JPQL**:

```
SELECT p.idPersona FROM Persona p;
SELECT MIN(p.idPersona), MAX(p.idPersona), COUNT(p.idPersona) FROM Persona p;
```

♦ **Diferencia clave:**

- **SQL** trabaja con columnas de tablas.
- **JPQL** trabaja con **objetos y atributos de entidad**.

2 Solución Paso a Paso

2.1 Crear la Clase PruebaJPQL

La clase `PruebaJPQL` ejecutará la consulta y mostrará los resultados.

```
package sga.cliente.jpql;

import java.util.List;
import jakarta.persistence.*;
import java.util.Iterator;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PruebaJPQL {

    private static final Logger log = LoggerFactory.getLogger(PruebaJPQL.class);

    public static void main(String[] args) {
        String jpql = null;
        List<Persona> personas = null;

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {
```

```
//1. Consulta de todos los objetos de tipo Persona
log.info("\n1. Consulta de todas las Personas");
jpql = "select p from Persona p";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//2. Consulta de una persona por ID
log.info("\n2. Consulta de Persona con ID = 1");
jpql = "SELECT p FROM Persona p WHERE p.idPersona = 1";
Persona persona = em.createQuery(jpql, Persona.class).getSingleResult();
log.info("Persona encontrada: " + persona);

//3. Consulta de una persona por nombre
log.info("\n3. Consulta de Persona por nombre 'Ivonne'");
jpql = "SELECT p FROM Persona p WHERE p.nombre = 'Ivonne'";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//4. Consulta de datos individuales (tupla)
log.info("\n4. Consulta de datos individuales: nombre, apellido, email");

jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
Iterator iter = em.createQuery(jpql).getResultList().iterator();

while (iter.hasNext()) {
    Object[] tupla = (Object[]) iter.next();
    String nombre = (String) tupla[0];
    String apellido = (String) tupla[1];
    String email = (String) tupla[2];

    log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
}

//5. Obtiene el objeto Persona y el id, se crea un arreglo de tipo Object con 2 columnas
log.info("\n5. Obtiene el id y el objeto Persona, se crea un arreglo de tipo Object con 2
columnas");
Object[] tupla = null;
jpql = "select p.idPersona, p from Persona p ";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
    int idPersona = (int) tupla[0];
    persona = (Persona) tupla[1];
    log.info("ID persona:" + idPersona);
}
```

```

        log.info("Objeto persona:" + persona);
    }

    //6. Consulta de todas las personas
    log.info("\n6. Consulta de los ids de todas las personas");
    jpql = "SELECT p.idPersona FROM Persona p";
    List<Integer> ids = em.createQuery(jpql, Integer.class).getResultList();
    ids.forEach(idPersona -> log.info("ID persona: " + idPersona));

    //7. Regresa el valor minimo y maximo del idPersona (scaler result)
    log.info("\n7. Regresa el valor minimo y maximo del idPersona (scaler result)");
    jpql = "select MIN(p.idPersona) as MinId, MAX(p.idPersona) as MaxId, COUNT(p.idPersona) as
Contador from Persona p";
    iter = em.createQuery(jpql).getResultList().iterator();
    while (iter.hasNext()) {
        tupla = (Object[]) iter.next();
        Integer idMin = (Integer) tupla[0];
        Integer idMax = (Integer) tupla[1];
        Long count = (Long) tupla[2];
        log.info("idMin:" + idMin + ", idMax:" + idMax + ", count:" + count);
    }
}

private static void mostrarPersonas(List<Persona> personas) {
    for (Persona p : personas) {
        log.info("Persona: " + p);
    }
}
}

```

3 Consulta de IDs en JPQL

3.1 Recuperar Solo los IDs de Todas las Personas

✦ Obtenemos solo el atributo `idPersona` de todos los registros.

```

String jpql = "SELECT p.idPersona FROM Persona p";
List<Integer> ids = em.createQuery(jpql, Integer.class).getResultList();
ids.forEach(idPersona -> log.info("ID Persona: " + idPersona));

```

🗨 Explicación:

- `SELECT p.idPersona FROM Persona p`: Recupera **solo el ID** en lugar del objeto completo.
- `createQuery(jpql, Integer.class).getResultList()`: Devuelve una lista de enteros en lugar de objetos `Persona`.
- `ids.forEach(idPersona -> log.info("🚩 ID Persona: " + idPersona));`: Imprime los IDs obtenidos.

📄 3.2 Obtener el ID Mínimo, Máximo y el Conteo de Registros

🚩 Utilizamos funciones escalares (`MIN`, `MAX`, `COUNT`) en JPQL.

```
String jpql = "SELECT MIN(p.idPersona), MAX(p.idPersona), COUNT(p.idPersona) FROM Persona p";
Iterator iter = em.createQuery(jpql).getResultList().iterator();

while (iter.hasNext()) {
    Object[] tupla = (Object[]) iter.next();
    Integer idMin = (Integer) tupla[0];
    Integer idMax = (Integer) tupla[1];
    Long count = (Long) tupla[2];

    log.info("🚩 ID Mínimo: " + idMin + ", ID Máximo: " + idMax + ", Total Registros: " + count);
}
```

🗨 Explicación:

- `MIN(p.idPersona)`: Obtiene el **ID más bajo**.
- `MAX(p.idPersona)`: Obtiene el **ID más alto**.
- `COUNT(p.idPersona)`: Obtiene el **total de registros**.
- `Object[] tupla = (Object[]) iter.next();`: Se guardan los valores en una **tupla** (arreglo de objetos).

4 Resultado Esperado

Si ejecutamos la clase `PruebaJPQL`, veremos en la consola:

6. Consulta de los ids de todas las personas

```
[EL Fine]: sql: ServerSession(412925308)--Connection(696165690)--SELECT id_persona FROM PERSONA
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - ID persona: 5
```

```
[main] INFO sga.cliente.jpql.PruebaJPQL - ID persona: 2

[main] INFO sga.cliente.jpql.PruebaJPQL - ID persona: 3

[main] INFO sga.cliente.jpql.PruebaJPQL - ID persona: 1

[main] INFO sga.cliente.jpql.PruebaJPQL -

7. Regresa el valor minimo y maximo del idPersona (scaler result)

[EL Fine]: sql: ServerSession(412925308)--Connection(696165690)--SELECT MIN(id_persona), MAX(id_persona),
COUNT(id_persona) FROM PERSONA

[main] INFO sga.cliente.jpql.PruebaJPQL - idMin:1, idMax:5, count:4
```

Esto confirma que **las consultas JPQL** están funcionando correctamente 🎯.

5 Conclusión

🎯 En esta guía aprendimos a realizar **consultas avanzadas en JPQL** para obtener solo los IDs de las personas y calcular valores escalares.

♦ Puntos clave:

- ✓ **JPQL** permite seleccionar atributos individuales para optimizar consultas.
- ✓ **Funciones escalares** (MIN, MAX, COUNT) permiten calcular valores en JPA.

🚀 ¡Ahora puedes aplicar estas técnicas en tus proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)